

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (Medium)

Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I Geferlin Boffy R (192571080) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANSWERS:

1) Normalization up to 3NF

StudentID	StudentName	CourseID	CourseName	FacultyName
101	John	C101	DBMS	Anitha
101	John	C102	Python	Ravi
102	Alice	C101	DBMS	Anitha

Normalization Issues

1. Student details are repeated.
2. Course details are repeated.
3. Faculty details are repeated.
4. Update Anomaly – Updating faculty information requires multiple changes.
5. Insertion Anomaly – Cannot insert a course without a student.
6. Deletion Anomaly – Deleting the last student may delete course information.

First Normal Form (1NF)

All attributes contain atomic values.

Second Normal Form (2NF)

Split the table into:

STUDENT

| StudentID | StudentName |

COURSE

| CourseID | CourseName | FacultyName |

ENROLLMENT

| StudentID | CourseID |

Partial dependency is removed.

Third Normal Form (3NF)

Separate Faculty information.

STUDENT

| StudentID | StudentName |

FACULTY

| FacultyID | FacultyName |

COURSE

| CourseID | CourseName | FacultyID |

ENROLLMENT

| StudentID | CourseID |

Transitive dependency is removed.

Conclusion

The database is now in Third Normal Form (3NF). Redundancy is removed and insertion, deletion, and update anomalies are eliminated.

```
mysql> SHOW TABLES;
+-----+
| Tables_in_universitydb |
+-----+
| course                  |
| enrollment              |
| faculty                 |
| student                 |
+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Student;
+-----+-----+
| StudentID | StudentName |
+-----+-----+
|          101 | John        |
|          102 | Alice       |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Faculty;
+-----+-----+
| FacultyID | FacultyName |
+-----+-----+
|          1 | Anitha      |
|          2 | Ravi        |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Enrollment;
+-----+-----+
| StudentID | CourseID |
+-----+-----+
|          101 | C101     |
|          102 | C101     |
|          101 | C102     |
+-----+-----+
3 rows in set (0.00 sec)
```

2)Employees in Multiple Departments

Problem Statement

A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach.

Tables Used

Employee

EmpID EmpName

1	John
2	Alice
3	David

Department

DeptID DeptName

10	HR
20	IT
30	Finance

Works

EmpID DeptID

1	10
1	20

EmpID DeptID

2 20

3 30

3 10

SQL Query

```
SELECT E.EmpName
FROM Employee E
JOIN Works W
ON E.EmpID = W.EmpID
GROUP BY E.EmpID, E.EmpName
HAVING COUNT(W.DeptID) > 1;
```

Output

EmpName

John

David

Justification

- JOIN combines the Employee and Works tables.
 - GROUP BY groups records based on each employee.
 - COUNT() counts the number of departments assigned to each employee.
 - HAVING COUNT(DeptID) > 1 displays only employees working in more than one department.
-

Conclusion

The query successfully identifies employees who are assigned to multiple departments using JOIN, GROUP BY, and HAVING.

Created Tables:

```
mysql> SHOW TABLES;
+-----+
| Tables_in_companydb |
+-----+
| department          |
| employee             |
| works                |
+-----+
3 rows in set (0.00 sec)
```

Employee Table:

```
mysql> SELECT * FROM Employee;
+-----+-----+
| EmpID | EmpName |
+-----+-----+
|      1 | John    |
|      2 | Alice   |
|      3 | David   |
+-----+-----+
3 rows in set (0.00 sec)
```

Department Table:

```
mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
|      10 | HR       |
|      20 | IT       |
|      30 | Finance  |
+-----+-----+
3 rows in set (0.00 sec)
```

Employees Working in Multiple Departments (Query Output):

```
mysql> SELECT E.EmpName
-> FROM Employee E
-> JOIN Works W
-> ON E.EmpID = W.EmpID
-> GROUP BY E.EmpID, E.EmpName
-> HAVING COUNT(W.DeptID) > 1;
+-----+
| EmpName |
+-----+
| John    |
| David   |
+-----+
2 rows in set (0.00 sec)
```

3) Candidate Keys and BCNF

PART A – CONTENT FOR WORD DOCUMENT

Given Relation

R(A, B, C, D)

Functional Dependencies

$A \rightarrow B$

$B \rightarrow C$

$A \rightarrow D$

Step 1: Find the Candidate Key

Find the closure of A.

$$A^+ = \{A\}$$

Using $A \rightarrow B$

$$A^+ = \{A, B\}$$

Using $B \rightarrow C$

$$A^+ = \{A, B, C\}$$

Using $A \rightarrow D$

$$A^+ = \{A, B, C, D\}$$

Since A^+ contains all attributes, **A** is the **Candidate Key**.

Candidate Key = A

Step 2: Check BCNF

BCNF Rule:

Every determinant must be a candidate key.

Check each Functional Dependency.

FD 1

$$A \rightarrow B$$

A is a Candidate Key.

Satisfied

FD 2

$$B \rightarrow C$$

B is **not** a Candidate Key.

Not Satisfied

FD 3

$$A \rightarrow D$$

A is a Candidate Key.

Satisfied

Step 3: Conclusion

Since

$$B \rightarrow C$$

violates BCNF,

The relation is NOT in BCNF.

Step 4: BCNF Decomposition

Split the relation into

Relation 1

$$R_1(B, C)$$

Relation 2

$$R_2(A, B, D)$$

Now every determinant is a candidate key in its respective relation.

Hence,

The decomposed relations satisfy BCNF.

Final Answer

Candidate Key

A

BCNF Status

The given relation is NOT in BCNF.

BCNF Decomposition

R1(B, C)

R2(A, B, D)

4) Views and Integrity Constraints

Problem Statement

A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

Views

A **View** is a virtual table created from one or more tables. It provides limited access to data, improving security by showing only the required information.

Integrity Constraints Used

- **PRIMARY KEY** – Ensures each record is unique.
 - **NOT NULL** – Prevents null values.
 - **CHECK** – Ensures valid data (e.g., balance cannot be negative).
-

Account Table

AccNo	CustomerName	Balance
101	John	50000
102	Alice	35000
103	David	42000

SQL Query to Create View

```
CREATE VIEW CustomerView AS
SELECT AccNo, CustomerName
FROM Account;
```

The view hides the **Balance** column, allowing customers to see only their account number and name.

Conclusion

Using **Views** and **Integrity Constraints** improves database security, prevents invalid data, and ensures data consistency.

Account Table:

```
mysql> SELECT * FROM Account;
+-----+-----+-----+
| AccNo | CustomerName | Balance |
+-----+-----+-----+
| 101   | John         | 50000.00 |
| 102   | Alice        | 35000.00 |
| 103   | David        | 42000.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

CustomerView Table:

```
mysql> SELECT * FROM CustomerView;
+-----+-----+
| AccNo | CustomerName |
+-----+-----+
| 101   | John         |
| 102   | Alice        |
| 103   | David        |
+-----+-----+
3 rows in set (0.00 sec)
```

View Created Successfully:

```
mysql> SHOW FULL TABLES
-> WHERE Table_type='VIEW';
+-----+-----+
| Tables_in_bankingdb | Table_type |
+-----+-----+
| customerview         | VIEW       |
+-----+-----+
1 row in set (0.01 sec)
```

5) Sales Database Normalization

Problem Statement

A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to **3NF** or **BCNF** to eliminate redundancy.

Original Sales Table

OrderID	CustomerName	ProductID	ProductName	Price
101	John	P101	Laptop	50000
102	John	P102	Mouse	800
103	Alice	P101	Laptop	50000

Problems in the Table

1. Customer details are repeated.
2. Product details are repeated.
3. Update Anomaly – Updating a product price requires multiple changes.
4. Insertion Anomaly – A new product cannot be added without an order.
5. Deletion Anomaly – Deleting the last order may remove product information.

First Normal Form (1NF)

- All attributes contain atomic (single) values.
- No repeating groups are present.

Second Normal Form (2NF)

Split the table into:

CUSTOMER

| CustomerID | CustomerName |

PRODUCT

| ProductID | ProductName | Price |

ORDERS

| OrderID | CustomerID | ProductID |

This removes **partial dependency**.

Third Normal Form (3NF)

The above three tables satisfy **3NF** because:

- Every non-key attribute depends only on the primary key.
- There are no transitive dependencies.

Conclusion

The database is successfully normalized up to **3NF**, reducing redundancy and eliminating insertion, update, and deletion anomalies.

Sales database Tables:

```
mysql> SHOW TABLES;
+-----+
| Tables_in_salesdb |
+-----+
| customer           |
| orders             |
| product            |
+-----+
3 rows in set (0.00 sec)
```

Customer table:

```
mysql> SELECT * FROM Customer;
+-----+-----+
| CustomerID | CustomerName |
+-----+-----+
|          1 | John         |
|          2 | Alice        |
+-----+-----+
2 rows in set (0.00 sec)
```

Product Table:

```
mysql> SELECT * FROM Product;
+-----+-----+-----+
| ProductID | ProductName | Price  |
+-----+-----+-----+
| P101      | Laptop      | 50000.00 |
| P102      | Mouse       | 800.00   |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Orders Table:

```
mysql> SELECT * FROM Orders;
+-----+-----+-----+
| OrderID | CustomerID | ProductID |
+-----+-----+-----+
|       101 |          1 | P101      |
|       102 |          1 | P102      |
|       103 |          2 | P101      |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

6) Joins and Subqueries

Problem Statement

Given a dataset of customers and orders, write SQL queries using **Joins** and **Subqueries** to retrieve meaningful information such as frequent customers.

Customer Table

CustID	CustomerName
1	John
2	Alice
3	David

Orders Table

OrderID	CustID	Product
101	1	Laptop
102	1	Mouse
103	2	Keyboard
104	1	Monitor

OrderID CustID Product

105 3 Printer

SQL Query Using JOIN

```
SELECT C.CustomerName, O.OrderID, O.Product
FROM Customer C
JOIN Orders O
ON C.CustID = O.CustID;
```

Output

CustomerName	OrderID	Product
John	101	Laptop
John	102	Mouse
Alice	103	Keyboard
John	104	Monitor
David	105	Printer

SQL Query Using Subquery

```
SELECT CustomerName
FROM Customer
WHERE CustID IN
(
SELECT CustID
FROM Orders
GROUP BY CustID
HAVING COUNT(*) > 2
);
```

Output

CustomerName
John

Conclusion

- **JOIN** combines customer and order information.
- **Subquery** identifies customers who placed more than two orders.

Customer Database Tables:

```
mysql> SHOW TABLES;
+-----+
| Tables_in_customerdb |
+-----+
| customer              |
| orders                |
+-----+
2 rows in set (0.00 sec)
```

Customer table:

```
mysql> SELECT * FROM Customer;
+-----+-----+
| CustID | CustomerName |
+-----+-----+
|      1 | John         |
|      2 | Alice        |
|      3 | David        |
+-----+-----+
3 rows in set (0.00 sec)
```

JOIN Query Output:

```
mysql> SELECT C.CustomerName, O.OrderID, O.Product
-> FROM Customer C
-> JOIN Orders O
-> ON C.CustID = O.CustID;
+-----+-----+-----+
| CustomerName | OrderID | Product |
+-----+-----+-----+
| John         | 101     | Laptop  |
| John         | 102     | Mouse   |
| John         | 104     | Monitor |
| Alice        | 103     | Keyboard|
| David        | 105     | Printer |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

Subquery – Frequent Customers:

```
mysql> SELECT CustomerName
      -> FROM Customer
      -> WHERE CustID IN
      -> (
      -> SELECT CustID
      -> FROM Orders
      -> GROUP BY CustID
      -> HAVING COUNT(*) > 2
      -> );
```

```
+-----+
| CustomerName |
+-----+
| John         |
+-----+
```

```
1 row in set (0.00 sec)
```

7) 4NF Decomposition

Problem Statement

A relation contains multi-valued dependencies. Analyze the structure and decompose it into **Fourth Normal Form (4NF)**, explaining the reasoning.

Given Relation

STUDENT

StudentID	PhoneNo	Skill
101	9876543210	Python
101	9876543210	Java
101	9123456789	Python
101	9123456789	Java

Analysis

In this relation:

- A student can have **multiple phone numbers**.
- A student can have **multiple skills**.
- **Phone numbers** and **skills** are independent of each other.

Therefore, the relation has **multi-valued dependencies**, causing redundancy.

Multi-Valued Dependencies

StudentID $\twoheadrightarrow$ PhoneNo

StudentID $\rightarrow\rightarrow$ Skill

4NF Decomposition

Split the relation into two tables.

STUDENT_PHONE

StudentID PhoneNo

101	9876543210
101	9123456789

STUDENT_SKILL

StudentID Skill

101	Python
101	Java

Reason for Decomposition

- Removes redundant data.
- Eliminates unnecessary combinations.
- Prevents update, insertion, and deletion anomalies.
- Satisfies **Fourth Normal Form (4NF)**.

Conclusion

The relation is successfully decomposed into **4NF** by separating the multi-valued dependencies into independent tables.

8) Integrity Constraints

Problem Statement

A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.

Problems in the Database

1. Duplicate Employee IDs.
 2. Empty employee names.
 3. Invalid age values.
 4. Negative salary values.
 5. Duplicate email addresses.
 6. Invalid department references.
-

Suggested Integrity Constraints

- **PRIMARY KEY** – Prevents duplicate Employee IDs.
- **NOT NULL** – Ensures employee names are not empty.
- **CHECK** – Ensures valid age and salary values.
- **UNIQUE** – Prevents duplicate email addresses.

- **FOREIGN KEY** – Maintains valid department references.

Conclusion

Applying these integrity constraints improves data consistency, prevents invalid entries, and maintains database integrity.

Company Database table:

```
mysql> SHOW TABLES;
+-----+
| Tables_in_companydb_q8 |
+-----+
| department              |
| employee                 |
+-----+
2 rows in set (0.00 sec)
```

Department Table:

```
mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
|      10 | HR       |
|      20 | IT       |
|      30 | Finance  |
+-----+-----+
3 rows in set (0.00 sec)
```

Employee Table:

```
mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+-----+-----+
| EmpID | EmpName | Age | Salary | Email | DeptID |
+-----+-----+-----+-----+-----+-----+
|      1 | John    | 25  | 30000.00 | john@gmail.com | 10 |
|      2 | Alice   | 28  | 35000.00 | alice@gmail.com | 20 |
|      3 | David   | 30  | 40000.00 | david@gmail.com | 30 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Employee Table Structure Showing Integrity Constraints:

```
mysql> DESC Employee;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| EmpID | int  | NO   | PRI | NULL    |       |
| EmpName | varchar(30) | NO |       | NULL    |       |
| Age    | int  | YES  |       | NULL    |       |
| Salary | decimal(10,2) | YES |       | NULL    |       |
| Email  | varchar(50) | YES | UNI | NULL    |       |
| DeptID | int  | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```


9) 5NF Decomposition

Problem Statement

For a given relational schema, analyze the presence of join dependencies and decompose it into **Fifth Normal Form (5NF)** to remove redundancy.

Given Relation

SUPPLIER_PROJECT_PRODUCT

Supplier Project Product

S1	P1	Laptop
S1	P2	Laptop
S2	P1	Printer

Analysis

The relation contains a **join dependency** because supplier, project, and product information are stored together.

This leads to:

- Data redundancy
 - Update anomaly
 - Insertion anomaly
 - Deletion anomaly
-

5NF Decomposition

SUPPLIER_PROJECT

Supplier Project

S1	P1
S1	P2
S2	P1

SUPPLIER_PRODUCT

Supplier Product

S1	Laptop
S2	Printer

PROJECT_PRODUCT

Project Product

P1	Laptop
P2	Laptop
P1	Printer

Conclusion

The relation is successfully decomposed into **5NF**, eliminating redundancy and join dependency.

10) SQL Query Optimization

Problem Statement

A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

Original Query

```
SELECT *  
FROM Employee, Department  
WHERE Employee.DeptID = Department.DeptID;
```

Optimized Query

```
SELECT E.EmpName, D.DeptName  
FROM Employee E  
JOIN Department D  
ON E.DeptID = D.DeptID;
```

Create Index

```
CREATE INDEX idxDept  
ON Employee(DeptID);
```

Advantages

- Faster data retrieval.
- Uses JOIN instead of old-style syntax.
- Index improves search performance.
- Retrieves only the required columns.

Conclusion

Using **JOIN** and **INDEX** optimizes query performance and reduces execution time.

Company database tables:

```
mysql> SHOW TABLES;  
+-----+  
| Tables_in_companydb_q10 |  
+-----+  
| department               |  
| employee                 |  
+-----+  
2 rows in set (0.00 sec)
```

Employee table:

```
mysql> SELECT * FROM Employee;
+-----+-----+-----+
| EmpID | EmpName | DeptID |
+-----+-----+-----+
|      1 | John    |      10 |
|      2 | Alice   |      20 |
|      3 | David   |      30 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Optimized JOIN Query Output:

```
mysql> SELECT E.EmpName, D.DeptName
-> FROM Employee E
-> JOIN Department D
-> ON E.DeptID = D.DeptID;
+-----+-----+
| EmpName | DeptName |
+-----+-----+
| John    | HR       |
| Alice   | IT       |
| David   | Finance  |
+-----+-----+
3 rows in set (0.00 sec)
```

Index Created Successfully:

```
mysql> SHOW INDEX FROM Employee;
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| Table      | Non_unique | Key_name | Seq_in_index | Column_name | |
| Collation  | Cardinality | Sub_part | Packed       | Null        | Index_type |
| Comment    | Index_comment | Visible | Expression    |             |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| employee   |            0 | PRIMARY  |              1 | EmpID       | |
| A          |            3 |          | NULL         | NULL        | BTREE     |
|            |            | YES      | NULL         |             |
+-----+-----+-----+-----+-----+-----+
| employee   |            1 | idxDept  |              1 | DeptID      | |
| A          |            3 |          | NULL         | YES         | BTREE     |
|            |            | YES      | NULL         |             |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```